CodeCapture: A Tool for Extracting Code and Workflow from Programming

Software Project Lab 3 Technical Report

Submitted by
Sifat Sikder
BSSE-1221
Institute of Information Technology
University of Dhaka

Supervised by
Mridha Md. Nafis Fuad
Lecturer
Institute of Information Technology
University of Dhaka

Submission Date: 1 December 2024



Letter of Transmittal

1 December, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **CodeCapture**: A Tool for Extracting Code and Workflow from Programming Tutorials

Sir,
I am submitting the technical report for our Software Project Lab 3, "**CodeCapture**: A Tool for Extracting Code and Workflow from Programming Tutorials."
This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Sifat Sikder
BSSE-1221
Institute of Information Technology
University of Dhaka

—----------

Supervisor's Signature

Abstract

In the modern era of online learning, programming video tutorials hosted on platforms such as YouTube, Coursera, and Udemy have become invaluable resources for developers of all experience levels. However, these tutorials often lack direct access to project codebases, forcing learners to recreate the code manually, a process prone to errors and inefficiencies. Additionally, understanding the coding workflow, which involves actions like entering, editing, and deleting code, can be overwhelming in large projects.

To address these challenges, **CodeCapture** is an innovative tool designed to automate the extraction of source code and workflows from programming video tutorials. By analyzing video tutorials frame-by-frame, CodeCapture captures the folder structure and codebase, reconstructing them for direct download. Beyond code extraction, the tool generates comprehensive coding workflows, extracts video summaries using deep learning models, and offers utilities for project zipping and selective file downloads.

CodeCapture enhances the learning experience by providing learners with organized source code, step-by-step workflows, and concise summaries, significantly reducing the learning curve, improving comprehension, and empowering developers to focus on coding rather than manual recreation tasks.

Table of Contents

1. Introduction **6**
1.1 Motivation 6
1.2 Scope 7
1.3 Project Timeline 7
1.4 Purpose of Document 7
2. Project Description **8**
2.1. Quality Function Deployment 8
2.1.1. Normal Requirements 8
2.1.2. Expected Requirements 9
2.1.3. Exciting Requirements 9
2.2 Usage Scenario 10
3. Scenario Based Modeling **12**
3.1 Use Case Diagram 12
3.1.1 CodeCapture 12
3.1.2 Modules of CodeCapture 13
3.2.2.1. Extract Source Code 13
3.2.2.2. Generate Workflow 14
3.2.2.3. Generate Summary 14
4. Class-Based Modeling **15**
4.1 Analysis Classes 15
4.2 Class cards 16
4.3 CRC Diagram 19
5. Architectural Design **20**
5.1 Architectural Context Diagram 20
5.2 Archetypes 21
5.3 Top Level Components 22
5.4 Mapping Requirements to Software Architecture 23
6. Block Diagram **24**
7. Preliminary Test Plan **25**
7.1 High Level Description of Testing Goals 25
7.2 Test Cases 25
Conclusion **30**

List Of Figures

Figure-1: Timeline for CodeCapture - 7

Figure-2: UseCase Level 0 (CodeCapture) - 12

Figure-3: UseCase Level 1 (Modules of CodeCapture) - 13

Figure-4: CRC Diagram of CodeCapture - 19

Figure-5: Architectural Context Diagram - 20

Figure-6: Archetypes - 21

Figure-7: Top Level Components - 22

Figure-8: Block Diagram of CodeCapture - 24

List of Tables

Table-1: CRC Card (VideoUploader) - 16

Table-2: CRC Card (VideoProcessor) - 16

Table-3: CRC Card (CodeExtractor) - 17

Table-4: CRC Card (WorkflowGenerator) - 17

Table-5: CRC Card (SummaryGenerator) - 18

Table-6: CRC Card (OutputDownloader) - 18

Table-7: Mapping of Components with Requirement - 23

1. Introduction

The rapid growth of online learning platforms has revolutionized how developers acquire programming skills. Video tutorials on platforms like YouTube, Coursera, and Udemy have become popular for learners seeking practical guidance on coding and software development. These tutorials often cover complex topics, making them invaluable resources for individuals at various skill levels. However, a recurring challenge in these tutorials is the lack of readily available project codebases, which forces learners to replicate code from scratch. This tedious and error-prone process not only wastes time but also detracts from the primary learning objective.

To address these gaps, CodeCapture introduces a novel approach to automate the extraction of source code and workflows from programming tutorials. By leveraging computer vision and deep learning, CodeCapture streamlines the learning experience, enabling users to focus on understanding concepts rather than recreating code. The tool delivers outputs such as organized project files, coding workflows, and summarized content, offering a comprehensive learning solution tailored for developers.

1.1 Motivation

The absence of easily accessible codebases and detailed coding workflows in online programming tutorials creates significant barriers for learners. Reconstructing code manually from tutorial videos is not only time-consuming but also prone to errors, often resulting in frustration and a diminished learning experience. Even when code is provided, the lack of clear step-by-step workflows makes it difficult for learners to understand how complex projects are developed.

This challenge highlights the need for a tool that can bridge the gap between tutorial content and practical learning. CodeCapture is motivated by the desire to simplify the learning process by automating code and workflow extraction directly from video tutorials. By providing structured, actionable outputs, CodeCapture empowers developers to grasp coding concepts more effectively, accelerate their learning, and ultimately enhance their productivity.

1.2 Scope

The scope of **CodeCapture** is given below:

- It will be a single page web application.
- It will extract the whole source code.
- It will reconstruct the code with directory hierarchy.
- It will generate a detailed workflow of coding actions (e.g., entering, deleting, selecting code)
- It will summarize the tutorial content.

1.3 Project Timeline

The estimated timeline of CodeCapture is given below:

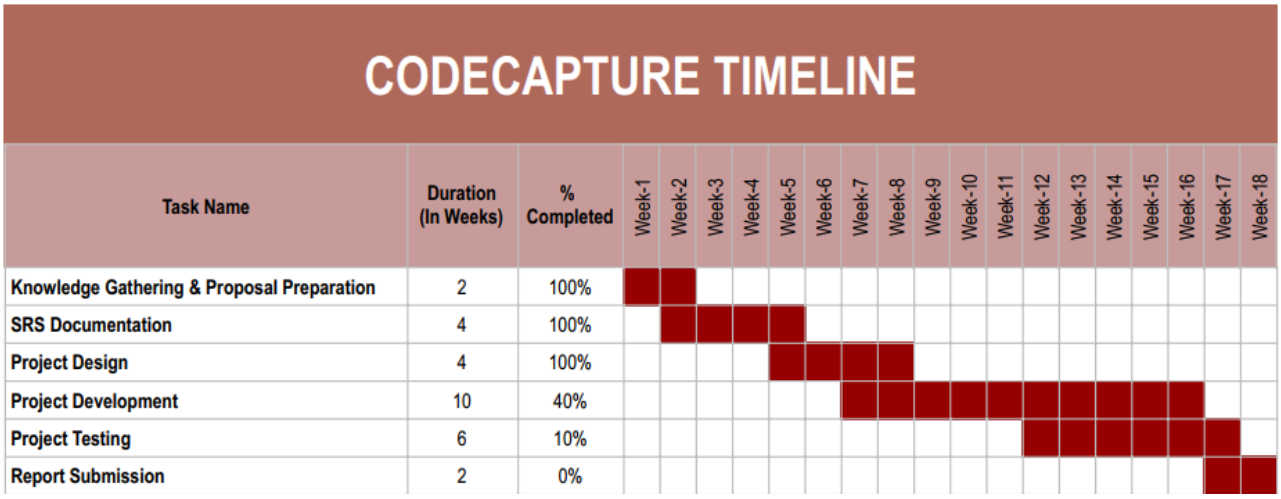


Figure-1: Timeline for CodeCapture

1.4 Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

CodeCapture is a tool designed to revolutionize the learning experience for developers by automating the extraction of source code and coding workflows directly from programming video tutorials. By leveraging advanced computer vision and deep learning techniques, CodeCapture processes tutorial videos frame-by-frame to extract source code and detailed coding workflows.

The tool addresses the common challenge of manually recreating codebases from tutorials, which is both time-consuming and error-prone. In addition to source code extraction, CodeCapture generates a step-by-step coding workflow to help learners understand the sequence of actions involved in project development. The tool also includes features like video summary generation, project zipping for simplified downloads, and selective file access.

CodeCapture's core functionality focuses on:

- Extracting source code accurately from video tutorials.
- Identifying coding actions like entering, deleting, and selecting code for workflow generation.
- Summarizing video tutorials into concise, actionable insights using deep learning models.

2.1. Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. With respect to this project, the following requirements are identified

2.1.1. Normal Requirements

1. **Source Code Extraction:**
    - Accurate extraction of code from the video tutorials.
    - Reconstruction of extracted code into an organized folder structure.
2. **Workflow Generation:**
    - Detection of coding actions (entering, selecting, deleting) for workflow generation.
    - Compilation of coding actions into a comprehensive step-by-step workflow.
3. **Project Organization and Accessibility:**
    - Automatic creation of a zip file containing the extracted codebase and folder structure.

2.1.2. Expected Requirements

1. **Intuitive User Interface:**
    - Simple and user-friendly interface to upload tutorials and view extracted outputs.
2. **Multi-format Input Support:**
    - Compatibility with multiple video formats (e.g., MP4, AVI, YouTube links) for input.
3. **Language Support:**
    - Compatibility with tutorials in multiple programming languages, such as Python, Java.

2.1.3. Exciting Requirements

1. **Summary Generation:**
    - Conversion of video content into text using speech recognition.
    - Generation of concise summaries using deep learning techniques.

2.2 Usage Scenario

"CodeCapture: A Tool for Extracting Code and Workflow from Programming Tutorials" will be a standalone web application designed to automate the extraction of source code, workflows, and summaries from programming video tutorials. By offering users direct access to structured project files and workflow guides, it will simplify the learning process and enhance the tutorial experience. The usage scenario for CodeCapture can be described as follows:

Accessing CodeCapture

Users will access CodeCapture through their web browsers, where they will be presented with an intuitive interface to start the process. They will either input the link to a YouTube video tutorial or upload a video file directly from their local storage. CodeCapture will be designed to handle various video formats, ensuring compatibility with a wide range of tutorials.

Video Preprocessing

Once a video tutorial is provided, CodeCapture will process it by converting the video into a set of frames, capturing one frame per second (1 fps). This approach ensures efficient processing while maintaining sufficient detail for analysis. The tool will then compare each frame with the previous one to detect changes in the visual content. Frames with no changes, such as static scenes or pauses in coding, will be discarded to streamline the analysis process and focus only on frames that contain relevant activity.

Extracting Source Code

During the source code extraction process, CodeCapture will first identify key elements in the video frames, including the sidebar, the active file name, and the code portion displayed within the active file. The sidebar will be analyzed to extract the folder hierarchy, which represents the project's structure as shown in the tutorial.

Once the folder hierarchy is established, the tool will focus on the active file and extract the code displayed in its corresponding portion of the frame. This extracted code will then be saved in the appropriate file within the identified folder hierarchy, ensuring the reconstructed project mirrors the structure and organization demonstrated in the video.

Generating Coding Workflow

CodeCapture will compare consecutive frames of the video to track changes in the code editor. It will detect actions such as entering new lines of code, modifying existing ones, or deleting sections. These actions will then be compiled into a detailed workflow guide, providing learners with a clear, step-by-step representation of the coding process.

Summarizing the Tutorial

To enhance the learning experience further, CodeCapture will extract the audio from the video and convert it into text captions using speech recognition technology. A deep learning model will summarize this text into a concise overview, highlighting the tutorial's main points, such as key coding techniques and implementation steps.

Downloading and Reviewing Outputs

After processing is complete, users will be presented with the final outputs, which will include the organized project folder, a detailed workflow guide, and a tutorial summary. Users will have the option to download the entire project as a zip file or selectively download specific files from the project folder directly through the web application.

Providing Feedback

To improve accuracy and adaptability, CodeCapture will include a feedback mechanism. Users will be able to report inaccuracies or issues they encounter in the extracted code, workflows, or summaries.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

3.1.1 CodeCapture

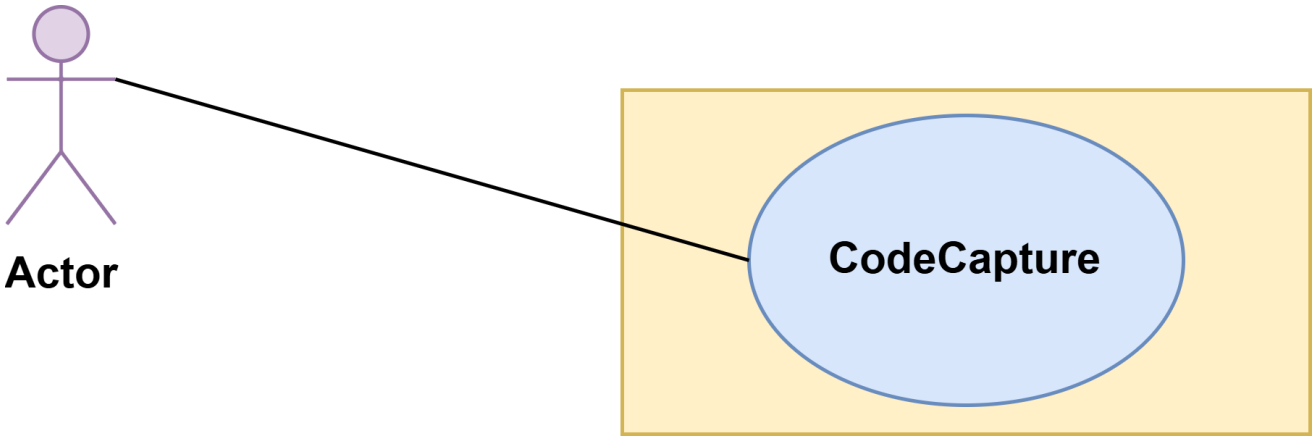


Figure-2: UseCase Level 0 (CodeCapture)

Primary actors: User
Goal in Context: The diagram shown above represents the entire automated tool for extracting source code and workflows from programming video tutorials – CodeCapture.

3.1.2 Modules of CodeCapture

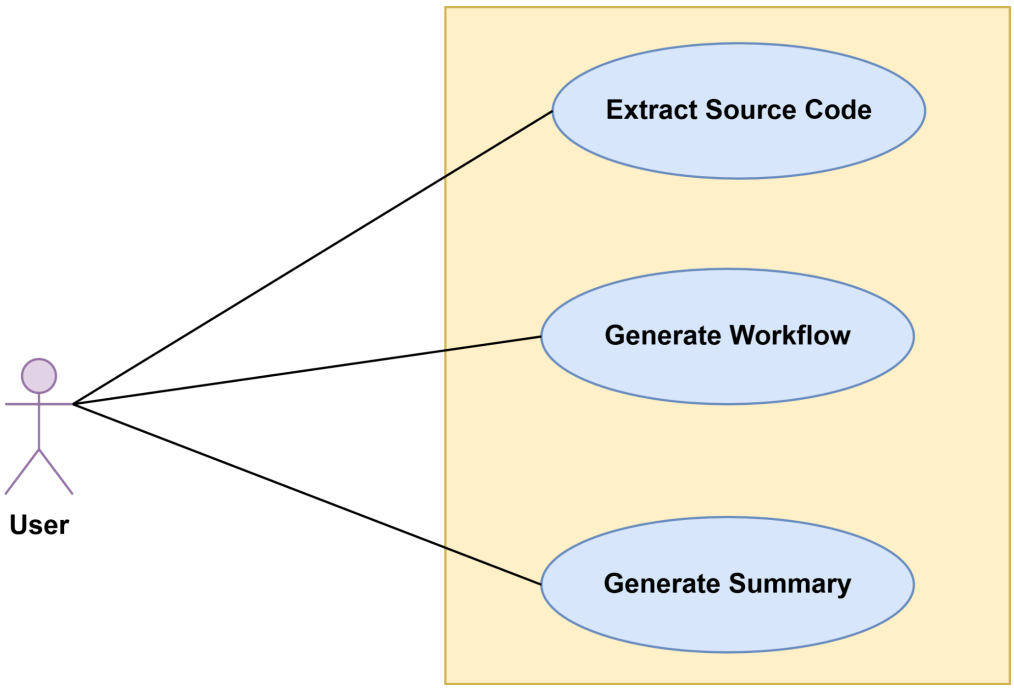


Figure-3: UseCase Level 1 (Modules of CodeCapture)

Primary Actor: User
Goal in Context: The diagram shows the modules of CodeCapture. CodeCapture consists of 3 modules. The modules can be elaborated as follows

3.2.2.1. Extract Source Code

The Extract Source Code module is responsible for identifying and extracting the coding content from the processed frames. It begins by analyzing the visible regions in each frame to detect key UI elements, such as the sidebar, the active file name, and the code editor area. The sidebar provides the folder hierarchy of the project, which is crucial for organizing the extracted code files. Then the module captures the textual content within the code editor area of the active file.

Once the folder hierarchy and active file are identified, the system saves the extracted code into its appropriate place within the identified structure. This ensures that the project's folder and file hierarchy reflect the original layout seen in the tutorial. By automating this meticulous process, the Extract Source Code module minimizes the effort required to reconstruct a project's codebase.

3.2.2.2. Generate Workflow

The Generate Workflow module is designed to reconstruct the sequence of coding actions performed in the tutorial. By comparing consecutive frames, it identifies regions of change within the code editor. Using action detection models, it classifies the detected changes into actions such as entering, selecting, or deleting code. This step-by-step tracking of user interactions creates a detailed and chronological coding workflow.

The module compiles these actions into a structured format that learners can follow, enabling them to understand how the code evolves over time. This feature is particularly useful for large projects, as it helps learners grasp the step-by-step development process.

3.2.2.3. Generate Summary

The Generate Summary module focuses on condensing the tutorial's content into a concise and informative overview. It first extracts audio from the video and applies automated speech recognition (ASR) to generate subtitles or captions. These captions are then processed through advanced transformer-based language models, which analyze and summarize the content of the tutorial.

4. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

4.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have derived the following analysis classes

- VideoUploader
- VideoProcessor
- CodeExtractor
- WorkflowGenerator
- SummaryGenerator
- OutputDownloader

4.2 Class cards

VideoUploader	
Attributes	**Methods**
- videoSource - videoType - validationStatus	+ validateInput(video) + uploadVideo(videoPath) + notifyProcessingStart()
Responsibilities	**Collaborators**
● Accepts video input from local files or YouTube links. ● Validates video format, type, and size to ensure compatibility. ● Prepares the video data for processing by saving relevant metadata.	● VideoProcessor ● SummaryGenerator

Table-1: CRC Card (VideoUploader)

VideoProcessor	
Attributes	**Methods**
- frameRate - extractedFrames - processedFrames	+ extractFrames(video, frameRate) + removeDuplicateFrames(frames) + prepareFramesForNextModule()
Responsibilities	**Collaborators**
● Converts videos into frames at a predefined rate (1 FPS). ● Filters frames to discard those with no changes, optimizing processing. ● Prepares and organizes valid frames for source code extraction.	● VideoUploader ● CodeExtractor ● WorkflowGenerator

Table-2: CRC Card (VideoProcessor)

CodeExtractor	
Attributes	**Methods**
- sideBar - fileHierarchy - activeFileName - extractedCode	+ extractSidebar(frames) + createfileHierarchy(sideBar) + extractCodeFromFrames(activeFrames) + saveCodeToFolderHierarchy()
Responsibilities	**Collaborators**
● Identifies key components in frames, such as sidebar and active file. ● Extracts code segments from the active file. ● Organizes the extracted code into an accurate file in the folder hierarchy.	● VideoProcessor ● OutputDownloader

Table-3: CRC Card (CodeExtractor)

WorkflowGenerator	
Attributes	**Methods**
- workflow - startFrames[] - endFrames[] - detectedActions[] - timestamps[]	+ detectCodingActions(frame1, frame2) + extractText(frame) + generateWorkflow()
Responsibilities	**Collaborators**
● Analyzes frame differences to detect coding actions such as entering, deleting, and editing. ● Creates a chronological workflow guide based on detected actions.	● VideoProcessor ● OutputDownloader

Table-4: CRC Card (WorkflowGenerator)

SummaryGenerator	
Attributes	**Methods**
- audio - transcribedText - summarizedText	+ extractAudio(video) + transcribeAudio(audio) + createSummary(transcribedText)
Responsibilities	**Collaborators**
● Extracts audio from the video for caption generation. ● Converts audio into captions using automated speech recognition (ASR). ● Generates a concise summary of the video tutorial from the transcribed text.	● VideoUploader ● OutputDownloader

Table-5: CRC Card (SummaryGenerator)

OutputDownloader	
Attributes	**Methods**
- codebase - workflow - summary - zippedOutput	+ combineOutputs() + prepareZipFile() + generateDownloadLinks()
Responsibilities	**Collaborators**
● Combines outputs from all modules into a unified deliverable. ● Provides downloadable links for the complete project or selected files. ● Prepares outputs as a compressed zip file for user convenience	● CodeExtractor ● WorkflowGenerator ● SummaryGenerator

Table-6: CRC Card (OutputDownloader)

4.3 CRC Diagram

The class cards of CodeCapture are given below:

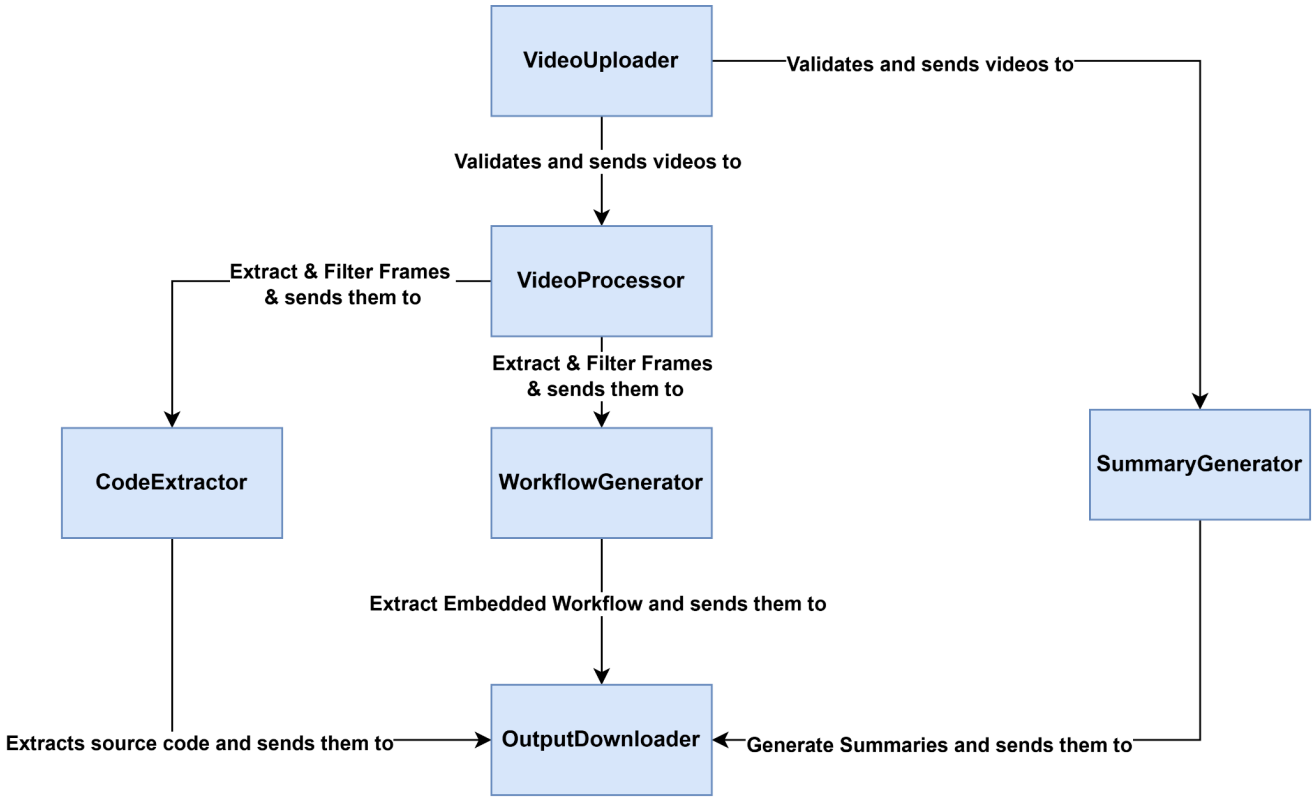


Figure-4: CRC Diagram of CodeCapture

5. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

5.1 Architectural Context Diagram

The architectural context diagram for the tool is given below

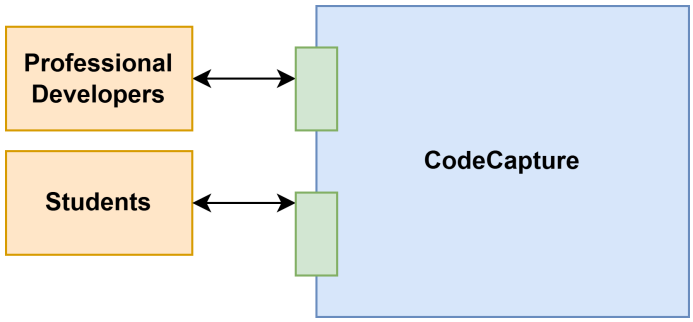


Figure-5: Architectural Context Diagram of CodeCapture

Two categories of users are supposed to be the target for the tool. They are professional software developers and students. Both types of users are the producers and also consumers of the information through the tool. There are no subordinate, superordinate or peer-level systems.

5.2 Archetypes

Client-server architecture is a common architectural pattern in software systems where the responsibilities and functionalities of the system are divided into two distinct roles: the client and the server. These roles interact with each other to provide services, process requests, and exchange data.

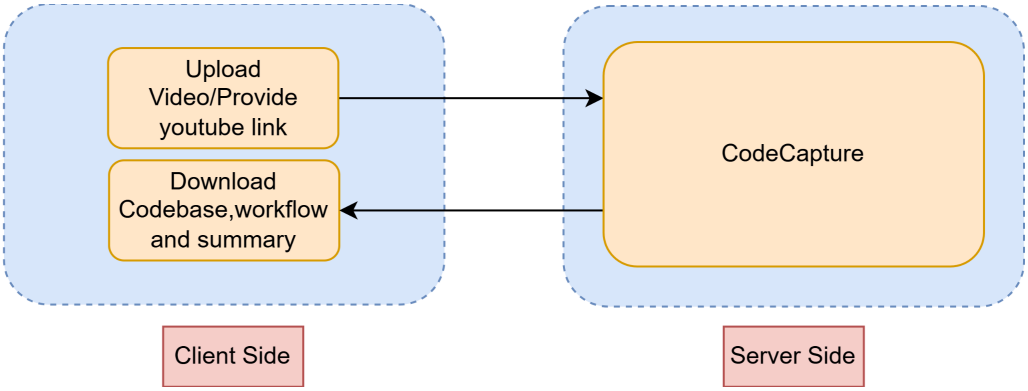


Figure-6: Archetypes of CodeCapture

5.3 Top Level Components

Overall architectural structure with top level components is illustrated below

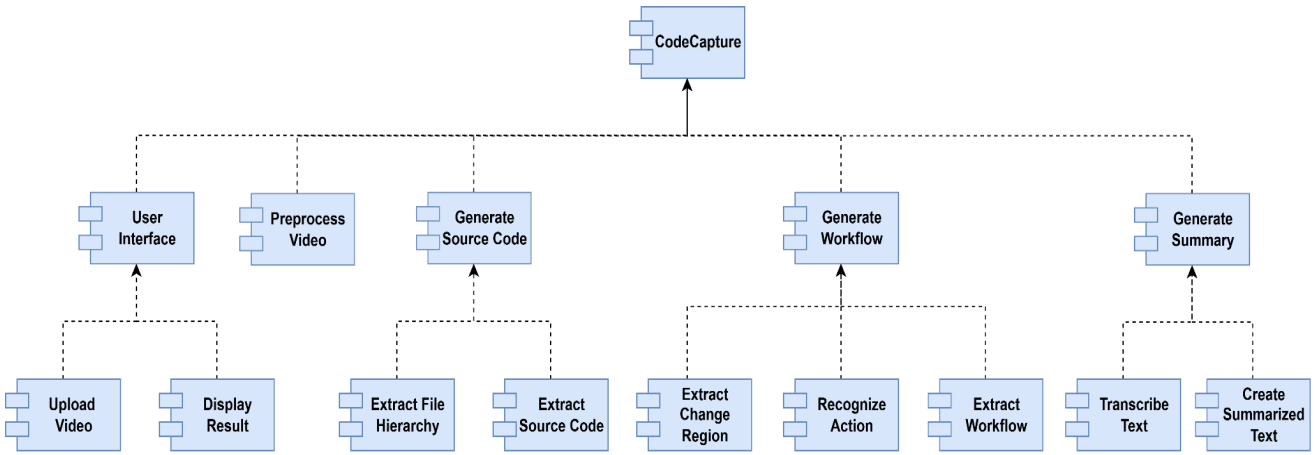


Figure-7: Top Level Components of CodeCapture

5.4 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown as below:

Requirements	Components
Accurate extraction of code from videos	Extract Source Code
Reconstruction into folder structure	Extract File Hierarchy
Detection of coding actions	Extract Change Region, Recognize Action
Compilation into step-by-step workflow	Extract Workflow
Conversion of video content into text	Transcribe Text
Generation of concise summaries	Create Summarized Text
Simple and user-friendly interface, Multi-format Input Support	Upload Video, Display Result
Compatibility with multiple video formats	Preprocess Video

Table-7: Mapping of Components with Requirement

6. Block Diagram

The block diagram illustrates the high-level architecture of CodeCapture. The tool operates in a systematic manner, consisting of the following stages:

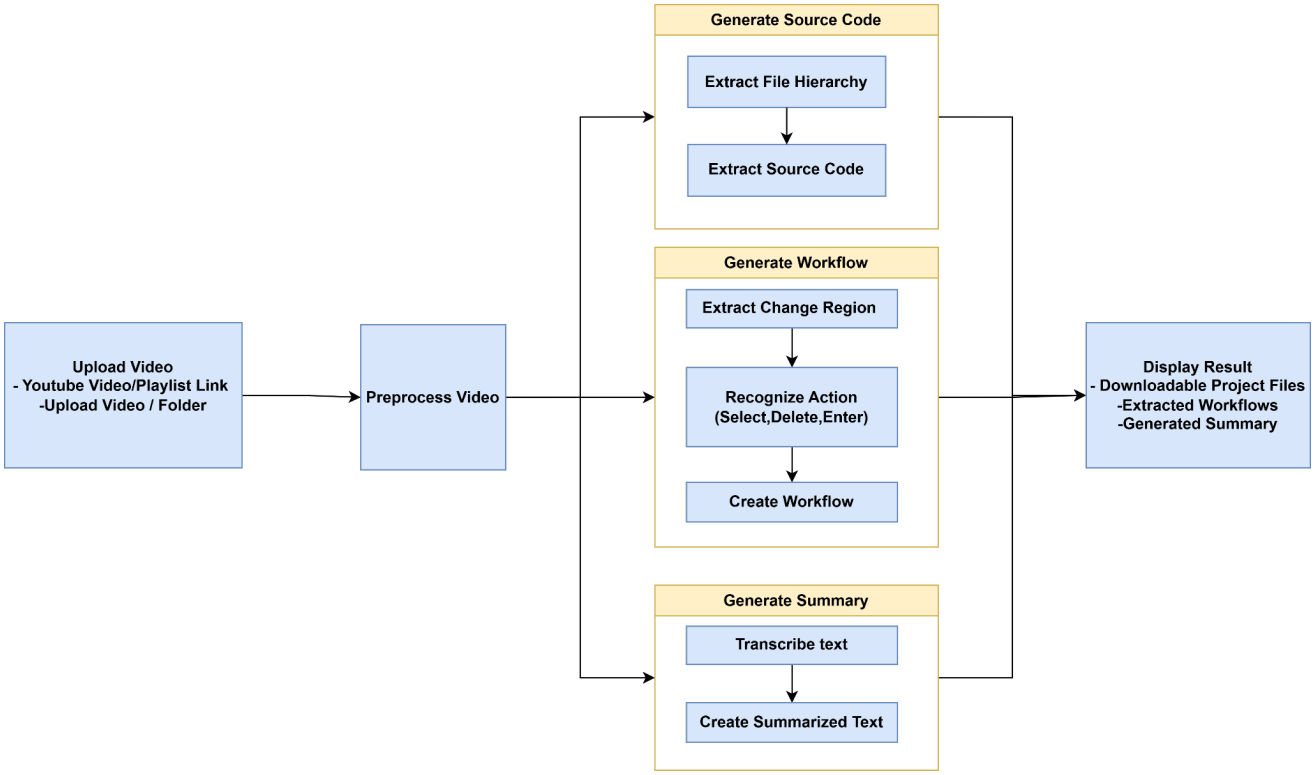


Figure-8: Block Diagram of CodeCapture

Upload Video: Users can upload videos directly, including YouTube links, playlists, or entire folders of video tutorials.

Preprocess Video: The uploaded video undergoes preprocessing to prepare it for subsequent stages of analysis.

Generate Source Code: This module extracts the file hierarchy and accurately captures the source code from the video, organizing it into a structured codebase.

Generate Workflow: CodeCapture detects code editing actions such as selecting, deleting, and entering text. These actions are used to create a step-by-step workflow for understanding coding practices.

Generate Summary: The system transcribes spoken content into text using advanced speech recognition and condenses it into concise summaries using deep learning.

Display Result: The final outputs include downloadable project files, extracted workflows, and summarized text, all presented to the user in a clear and accessible format.

7. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

7.1 High Level Description of Testing Goals

The testing goals for CodeCapture are as follows:

- To demonstrate that CodeCapture meets its functional requirements.
- To ensure that videos are processed accurately to extract source code and workflows.
- To verify the proper detection of change regions, coding actions (such as select, delete, and enter), and workflow generation.
- To ensure that the tool generates correct and coherent summaries from transcribed text.
- To ensure that the extracted workflows and code are organized and downloadable in a structured format.
- To ensure that CodeCapture handles multiple video formats and languages efficiently.
- To verify that the tool runs smoothly without crashes or significant delays.

7.2 Test Cases

Test cases for the testing of this tool are documented below-

Test Case ID	**T1**
Title	Web Application Access and Setup
Test Steps	1. Open the web application URL in a browser. 2. Wait for the web application to load without any errors. 3. Check if the page is responsive.
Expected Outcome	1. The web application should load without any errors. 2. The interface should be fully responsive.

Test Case ID	T2
Title	Generate Source Code for a Single Video
Test Steps	1. Run the extract source code option in the tool after uploading a video 2. Check if the source code is extracted and organized into the correct folder structure.
Expected Outcome	1. The extracted source code should be organized according to the folder structure of the video content. 2. Code snippets should be correctly formatted.

Test Case ID	T3
Title	Generate Source Code for a Playlist
Test Steps	1. Upload a playlist of multiple videos instead of a single video. 2. Extract source code, workflows, and summaries for the entire playlist.
Expected Outcome	The tool processes the playlist seamlessly, treating each video as a separate unit. Then It provides combined outputs as a whole.

Test Case ID	T4
Title	Generate Workflow
Test Steps	1. Run the workflow generation option in the tool after uploading a video. 2. Check if the system detects coding actions (e.g., select, delete, enter) properly.
Expected Outcome	1. Coding actions like selecting, deleting, and entering should be recognized and extracted correctly. 2. The generated workflow should be a clear, step-by-step sequence of actions performed in the video.

Test Case ID	T5
Title	Invalid Youtube Link
Test Steps	Provide a invalid or private YouTube video link in the tool.
Expected Outcome	The system should identify the invalidity of the provided link and notify the user (e.g., “The provided link is invalid.”).

Test Case ID	T6
Title	Download Project Files
Test Steps	After the video has been processed, click on the download button to download the project files (code, workflow, summary).
Expected Outcome	1. The zip file containing all the extracted code, workflow, and summary should be downloadable without errors. 2. The downloaded files should be correctly organized.

Test Case ID	T7
Title	Dynamic Code Editor Layout
Test Steps	Upload a video where the code editor layout frequently changes (e.g., the sidebar moves from left to right).
Expected Outcome	CodeCapture should adapt to the changing layout and correctly extract the folder structure and active code file.

Test Case ID	T8
Title	No Code Content
Test Steps	Upload a video tutorial that contains only a spoken explanation with no visible code or editor on the screen.
Expected Outcome	The system should identify the lack of visible code and notify the user (e.g., “No code detected in the video.”).

Test Case ID	T9
Title	Dynamic Code Changes in a Live Environment
Test Steps	Upload a video where the instructor frequently switches between different files or code bases in a live development environment.
Expected Outcome	CodeCapture correctly segregates code into separate files or projects.

Test Case ID	T10
Title	Extract File Hierarchy with Missing Parent Folder
Test Steps	1. Upload a video where only half of the file sidebar is visible, and the parent of some folders is not displayed. 2. Run the tool to extract the file hierarchy from the video. 3. Check the extracted file hierarchy for placeholders in the positions of the missing parent folders.
Expected Outcome	1. The tool accurately extracts the visible portion of the sidebar. 2. For missing parent folders, the tool inserts a placeholder (X) in their positions in the file hierarchy. 3. The extracted hierarchy maintains its structure with placeholders marking the missing information.

Conclusion

In this report, I have discussed all the necessary technical details to develop “CodeCapture: A Tool for Extracting Code and Workflow from Programming.” I have outlined the scope, assumptions, Quality Function Deployment (QFD), and usage scenarios. Scenario-based modeling and class-based modeling have been performed, along with a detailed architectural design. Additionally, a high-level description of testing goals and a summary of test cases have been presented to ensure the system's robustness and usability. Figures and tables have been included to enhance understanding and provide clarity regarding the technical details.

While CodeCapture addresses the innovative need for converting video tutorials into well-structured code and workflows, there is potential for further improvements. Future development could include support for additional programming languages, and the integration of advanced summarization and natural language processing techniques.

With this document, I have attempted to provide a clear and comprehensive understanding of the system's workflow and technical details. I hope this report serves as a valuable resource for anyone seeking insight into the design, functionality, and development of CodeCapture.